Table of Contents

Chapter1: Introduction	1.1
Chapter2 Network Service Infrastructure	1.2
Chapter3: TC Overview	1.3
Chapter4: Queueing Disciplines	1.4
Chapter5: Complex Egress Topologies	1.5
Chapter6: Using Explicit Classifiers	1.6
Chapter7: Classifier Action Blocks	1.7
Chapter8: Hardware Offloading	1.8
Chapter9: Kernel + User Code Overview	1.9
Chapter10: References And Footnotes	1.10

1. Introduction

Linux Traffic Control (TC) is a flexible, policy-driven packet processing architecture that incorporates a lot of interesting computer science algorithms which make it powerful at the expense of sometimes appearing complex.

In this document, we are going to take a top-down approach to explaining TC. We intend to describe TC incrementally, starting with a general high level over view targeted at the impatient, build concepts along the way, and lead all the way to accommodating readers who are interested in writing code. We assume no knowledge of TC in our initial section and introduce terminology and constructs that we believe will help the uninitiated grasp the concepts involved. If you have used TC more than once, you may want to skip the initial 1 or 2 sections.

The general breakdown of the document is as follows:

- First, we introduce some basic system constructs as a contextual aid to describe the TC architecture.
- We then proceed to map these system constructs to the TC architecture.
- Finally, we move to an incremental introduction of TC components; starting with basic leading all the way to the nuts and bolts details.

2. Network Service Infrastructure

In this section we are going to outline architectural concepts that will provide us context to describe TC architecture.

The TC architecture can be best described as a Network Service Infrastructure. We define a **Network Service** as:

> *The treatment of network packets, as defined by a user policy, so as to **achieve a defined goal on selected packets.***

Achieving the defined goal is the essence of a network service. To this end, the TC architecture provides infrastructure for the network operator to create policy graphs composed of several types of packet processing blocks.

All network services are composed of 1) *a discriminator to select the packets* and then 2) *one or more treatments to act on the selected packets*.

To build our narrative, let us start with an example of a simple network service, *firewalling*. The defined goals of a simple firewalling service are to select packets and drop the unwanted or accept the allowed.

Figure1-1 shows a graph of that can be used to describe a network service.

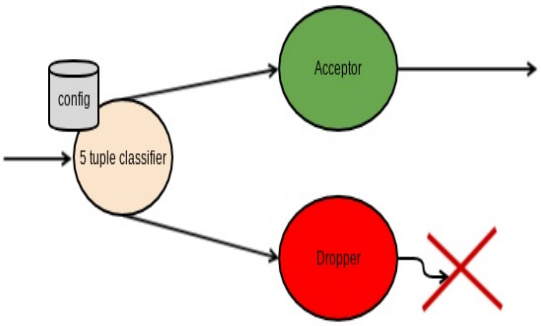


Figure1-1: Firewall Service

The service work flow is policy defined as follows:

1. To select bad/good packets based on a 5 tuple classification {source IP, destination IP, protocol(tcp/udp/sctp), source port, and destination port}
2. Depending on the policy from #1, execute an action to drop or accept the packet.

Let us build up this narrative to a more complex service. An example service builds along the definition in *Expedited Forwarding* behavior defined in RFC 3246. Figure1-2 shows a graph of that can be used to describe this packet service.

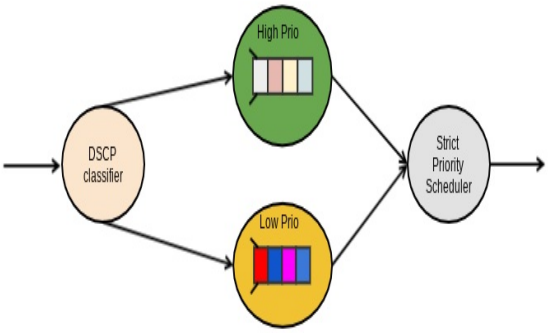


Figure1-2: More complex Service

The flow of this packet service is:

1. Using the packet's DSCP value, discriminate between voice packets and file transfer packets
2. Voice packets are enqueued on a high priority queue and file transfer traffic ends on a low priority queue.
3. A strict priority work-conserving scheduler is used to schedule out packets. The high priority queue is serviced ahead of the low priority queue, always.

2.1. Functional Block (FB)

One important detail to observe from the above examples is that a network service is a policy composition of several packet processing functions. We are going to refer to each of these packet processing functions as a ***Functional Block (FB)***.

[**XXX: Explain that we are extrapolating from RFC 3549 terminology where a FEC now maps to an FB**].

Figure1-3 illustrates an abstraction of a sample FB.

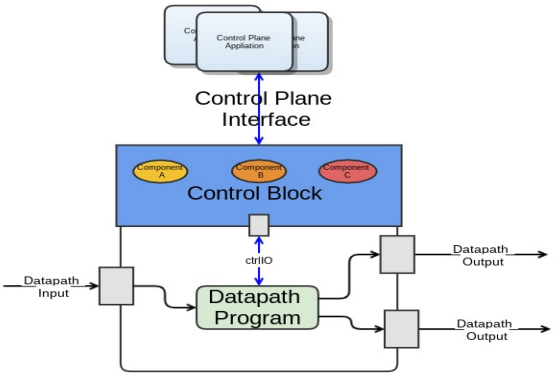


Figure1-3: Functional Block Abstraction

Lets dissect the different parts abstraction in Figure1-3:

- A *control* block which has:
  - An interface used for communication with control applications. Messages flow north-south between control applications and the *FB* control block.
  - A collection of objects labelled as "components" that are targetted by the control applications and used by the datapath.
- A *data* path(white block below control block) which has:
  - Datapath Packets and metadata flow in east-west via one or more "inputs" and leave the FB via one or more "outputs". Note on input/outputs[3]: different *FB* abstractions have different counts of

these elements. For example a classifier will have 1 input and N outputs (each to select different actions based on matches) whereas a drop action has 1 input and 0 outputs, etc.
- A datapath program that massages packets+metadata going into "inputs". Data and/or metadata may be consumed altogether, modified, extended, shrunk, etc before being sent out on "outputs".

NOTE: While we are going to use the concept of an *FB* to build up the context to describe TC in this document, we hope it is clear to the reader that the FB abstraction applies equally to many linux interfaces (example linux netdevices).

The control block data(objects) are shared by both the datapath and the control path.

2.1.1. FB Control Path

The control path applications talk to the control block of an *FB* using some control interface. To correlate to Linux TC, think of *iproute2* utilities (tc, ip, etc) residing in user space as the control applications and Netlink as the interface to the control block which resides in the kernel (although other system calls like ioctl, sysfs etc equally apply as control interfaces).

The Control block objects (Figure1-3 *A*, *B* and *C*) could be atomic types (example a 32 bit field, etc); complex data structures, variations of table abstractions(arrays, hashes, etc) etc. To correlate to TC - think of these as netlink attributes.

There are generally 3 types of semantics of in the interaction between the control applications and control components which help define the nature of the components:

1. **Control components that dictate data path policy.** This class of control components are provisioned by control applications and used by the datapath to make its processing decision. These types of components also hold runtime state and can be retrieved at runtime for analytics, etc. Examples of this class of components:
   - A policy table control component such as a filter hash table that is updated by a control plane and used in the data path to classify packets (example creating a new tc flower entry).
   - A boolean field to turn on a specific feature (example for a bridge FB "turn on learning". -etc
2. **Read-only control components which advertise capabilities and capacities.** Such control components are retrieved by the control plane to aid it in selecting the correct resources to achieve a desired service goal. Examples that fit this class of components:
   - Table sizes to describe table capacities.
   - A boolean component which is set to true by the datapath implementation to indicate it is capable of offloading a feature, etc.
   - bitmasks where a 1 value implies a supported feature (example, the linux netdev *features* 32-bit flags field) and a value of 0 implies the specific feature is not supported.
   - etc
3. **Components which define event subscription/publication rules.** Such control components are used by the control applications to subscribe to events; and by the datapath as controls for publishing event. Examples that fit this class of components:
   - a bitmap that is advertised by the *FB* to describe events that can be subscribed-to by control applications (example, this technique is very common in netlink interactions which apply to TC and many other *FB* types; see linux *rtnetlink_groups* as defined in "include/uapi/linux/rtnetlink.h" for examples). The control applications subscribe by setting the approriate flag bits.
   - etc

FB Control Interaction

There are two types of interactions that occur between a control application and the *FB* control block:

- A control application issues requests to the *FB* via the control interface directed at the control block components. These requests are typically commands with semantics like *create/update*, *get* and *delete* targetted at the control components. The *FB*, after processing, issues back responses to these command requests. Example, a control application could issue a request to dump a table, update a scalar component, create a table entry, retrieve a complex data structure component, etc.
- A control application issues event subscriptions to the *FB* via a control interface. When event conditions are met, the *FB* broadcasts these events to all subscribing applications. An example of an event that can be subscribed-to is an "*FB* table create" event which is published whenever a table component undergoes a change.

For a TC analogy - we point to the *iproute2::tc* command line utility as an example of a control application. This tool uses *libnetlink library* included in the *iproute2* package to mediate between user requests (via an API) and the control blocks in the kernel. One could envision other tools using the same library to interface to kernel control blocks (routing daemons like FRR or BIRD for example apply here).

2.1.2. FB Data Path: Types, Kinds and Instances

An *FB* datapath is defined uniquely based on several attributes:

- **What kind of packet data and associated metadata the *FB* is willing to accept in the data path and what it outputs after processing(east-west).** Figure1-3 illustrates an *FB* with one input and two possible outputs. The input into the data path block expects specific packet data and metadata (i.e a certain input signature is expected) and the outputs will emit specific packet data and metadata. This architecuteral choice allows us to define many types of *FBs* (example TC classifiers, actions, etc), each of which has well defined input/output signatures. We refer to each definition as an *FB Type (FBT)*.

  > *An FBT is a class template for implementing a defined Functional Block's processing goals.*

- **Datapath program implementation.** A specific *FBT* may have different implementations for differing packet processing desires. We refer to this implementation differentiation as an *FBT kind*. Example of TC *Classifier FBT* kinds are "flower", "u32", "fw", etc. Two different *FBT Kinds* will typically have very different control block components.

  > *A Kind is an FBT speacilization.*

- **Where and how an *FBT* runtime can be instantiated**. When instantiated, an FBT kind can be "hooked" to a certain "location" in the network stack (which also dictates its input/output signatures). Examples of TC qdisc "location" are netdevices "ingress", "egress", etc. We refer to an instantiated *FBT Kind* as a *Functional Block Instance (FBI)*. At instantion, an *FBI* is given a runtime identifier (typically a 32 bit number) to enable the control plane to uniquely address it.

  > *An FBI is a class instantiation of an FBT Kind.*

2.2. Packet Service

A *Packet Service* is described by a policy composition of *FBIs*. Any such FB composition can be best described from a graph point of view; packets flow from one *FBI* to other *FBIs* per policy definition with an end result of accomplishing the prescribed service.

The first *FBI* in a graph, anchored at some "location", often serves to discriminate traffic - this is then followed by a series of other *FBIs*; some FBs could constitute simple processing functions like dropping packets or enqueueing packets onto a selected queue; and yet others entail more complex scheduling algorithms for preferentially dequeueing packets out of several available queues before sending out an interface.

Figure1-4 shows a runtime graph of *FBIs* that can be used to describe a network service.

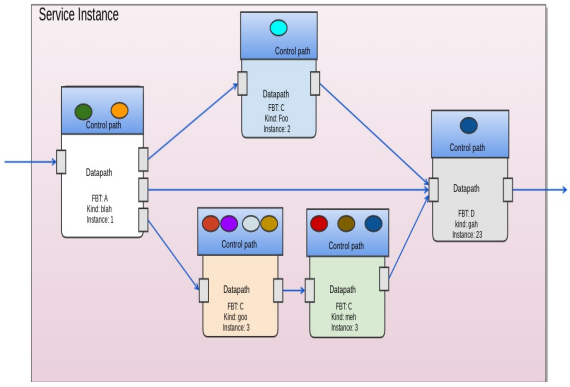


Figure1-4: Sample Service Graph

Figure1-4 shows a graph constituing 5 *FBIs* made up of 3 *FBTs*: A, C, and D. The different *FBT Kinds* are color encoded.

At each *FBI* node, a defined node *identifier* is also shown (Example instance *1* on the first *FBI* with *FBT* A, kind *blah*) As mentioned earlier, the node identifier represents an instantiation of the *FBT Kind* along with its runtime configuration attributes. We are going to interchangeably refer to this *identifier* as a *handle*. In Figure1-4, there are three *FBTs*: C (with Kinds *foo*, *goo* and *meh*), and one of each of A(kind *blah*) and D (kind *gah*). To provide analogy to TC, one could look at *FBI* A instance 1 as a classifier of Kind "u32"; *FBIs* of type C as TC actions and FBT D as a qdisc.

2.2.1. Describing Service Graphs

The graph links (edges) connecting the *FBI* nodes (vertices) represent packet flow direction; as an example packets could flow from node A/blah/1 to either C/foo/2, C/goo/3, or D/gah/23 depending on a processing decision made at node A/blah/1.

At this point we introduce a new term:

> A ***Service Topology*** is a composition of *FBIs* with an end goal of creating a Network Service. A *Service Topology* is modelled as a graph of *FBIs*. Figure1-4 shows a Service Topology.

Our earlier firewall example also provides a service topology which constitutes a classifier *FBI* , followed by a *FBI* to drop, or an *FBI* to accept a packet.

To provision a service policy, an admin/operator would create *FBIs* and stitch them together to the desired packet flow goals. This is done by a control application. A TC equivalence to creating these operations is achieved via the *tc* utility creating different *FBIs* and binding them together.

Let us construct the hypothetical service illustrated in Figure1-4 to demonstrate how such a service topology would be provisioned.

> 1. Create FBT A, kind blah, handle 1 with component attributes x="123", etc
> 2. Create FBT C, kind Foo, handle 2 with component attributes... and bind to parent FBI A/blah/1
> 3. Create FBT C, kind goo, handle 3 with component attributes... and bind to parent FBI A/blah/1
> 4. Create FBT C, kind meh, handle 3 with component attributes... and bind to parent FBI C/goo/3
> 5. Create FBT D, kind gah, handle 23 with component attributes... and bind to parent FBI A/blah/1

Listing1.1: Sample Service Creation

Step 1 in Listing1.1 *creates* the root node of the tree using FBT A Kind blah and gives it a handle id of 1. The creation process in this step (and all subsequent steps) also parameterizes the node attributes necessary for the node's instantiation, i.e. this is a process of instantation of the Functional Block Kind.

Step 2 shows both a create and bind/stiching operation: an FBT C, kind Foo with a handle id of 2 is created and is then bound/attached as a branch off the node FBI A/blah/1 created in step #1. In other words, FBI A/blah/1 is the parent of FBI C/Foo/3 (in the graph). Note: The creation and binding steps in #2-4 could have been broken down into two steps each i.e first a create then a bind step (for example one could create TC actions then bind them to one or more TC classifier filters).

At this point we are going to introduce another new term useful for defining service graphs:

> A ***Parent ID*** is a handle id of the node which the current specified node is branched from. As an example, from the listing above, the *FBI* A/blah/1 is the parent of nodes C/goo/3, C/Foo/2 and D/gah/23.

The points to take away here are: in order to build a policy graph (as illustrated in Figure1-4), each vertex in the graph needs an ID (which we refer to as a *handle*) and has to be bound/attached to a location in the graph which also has an ID (that we termed as *Parent ID*).

In the next section we will use and show how to map these concepts to TC.

3. TC Overview

In this section we will use terminology developed in the previous chapter to describe the basics of TC.

The Linux Traffic Control (TC) subsystem consists of 4 FBTs which are used in packet service creation. Each FBT has more than one ***kind***.

1. **Queuing disciplines (qdisc)**. A qdisc provides an *FBT* for queue algorithms (enqueuing and dequeuing packets). Some kinds of qdiscs include:
   - *Pfifo*, which implements a basic packet counting FIFO queueing algorithm.
   - *RED*, which implements the Random Early Detection (RED) algorithm.
   - *DRR*, which implements the Deficit Round Robin (DRR) algorithm.

   Note: qdiscs don't necessarily need to have queues as will be discussed later but will have enqueue inputs and dequeue outputs.
2. **Classifiers**. Classifiers provide *FBTs* that define filtering algorithms (to discriminate/select packets). Some kinds of classifiers include:
   - *u32*, which implements a 32-bit key/mask (ternary, lpm and exact) matching algorithm.
   - *flower*, which implements a multi-tuple matching algorithm.
   - *fw*, which implements a (skbmark) metadata based matching algorithm.
3. **Actions**. Actions provide *FBTs* for arbitrary packet processing. Some kinds of actions include:
   - *gact*, which implements amongst other things dropping and accepting of packets.
   - *mirred*, which implements redirecting or mirroring of packets.
   - *skbedit*, which implements metadata editing on a packet.
4. **Classes**. Classes provide templating for encapsulating qdisc FBTs to allow service topology branching. On their own classes do not implement algorithms; rather class instances serve to proxy access to the internal qdiscs they carry.

3.1. TC Functional Block Location And Identity

As discussed earlier, in order to build a service topology, each node in the topology graph needs two identifiers:

> - A ***handle ID***.
> - Attachment point on the topology graph which we defined as a ***parent ID***.

The TC architecture provides 32 bits for handle IDs. Throughout this document we may interchangeably refer to the *handle ID* as just *handle* and *parent ID* as *parent*.

The starting point to any service topology is an *anchor* location (A root in a tree). The anchor location is *where packets enter the topology*. At the moment, TC anchors its service topologies on a network port.

(**XXX: make this a footnote?**) A network port is a connectivity abstraction which may be physical or virtual. Infact a network port fits fine under the auspicies of an *FBT* description. In the Linux world a port is often refered to as a network device (or *netdev*) and in the SNMP world it is sometimes referred to as an *interface* - throughout this document we will use any of those terms (*port, device, netdev, and interface*) interchangeably to refer to this abstraction.

As mentioned earlier, at the moment, all anchor locations for a TC service topology start at a netdev; therefore the primary coordinate of a service topology must identify a network device.

There are several TC graph anchor locations as shown in Figure2.1.

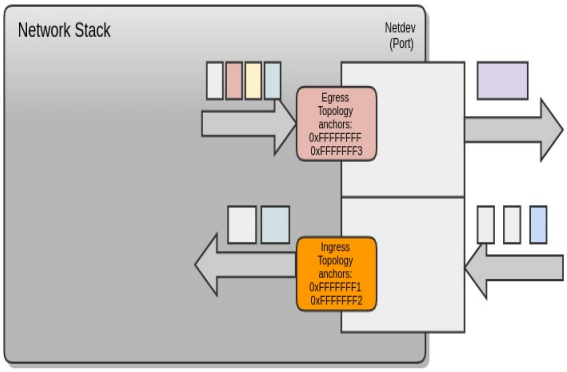


Figure2.1: Qdisc Attachment Points

Figure2.1 illustrates 4 handle IDs and 2 flow directions per netdev which are used to define service topologies.

Using packet flow directions: an egress topology is attached at the outgoing point of a netdev whereas an ingress topology is attached to the incoming point of a netdev. The first node on service topology graph in TC is always a qdisc FBT.

There are four IDs which are reserved on each netdev for the purpose of anchor location (root/starting point of a service graph) identification:

- An ID of 0xFFFFFFFF is reserved for use as a handle on the egress anchor point of a netdev as shown in Figure2.1. We refer to this as the anchor point of the ***EGRESS topology***. This location ID is also, for historical reasons, refered to as **root** in TC nomenclature. We are going to show later only qdiscs which encapsulate queues (of which there are numerous) can be attached as the first node on this topology.
- An ID of 0xFFFFFFF3 is reserved for use as a handle on the egress anchor point on the netdev for the ***EGRESSCLSACT topology***. We are going to show later that only the **clsact** qdisc can be bound as the first node to this anchor point. It should be noted that a packet flow may make use of both **EGRESS** and **EGRESSCLSACT** since they are different hooks and configured/controlled independently.
- An ID of 0xFFFFFFF1 is reserved for use as a handle on the ingress anchor point of a netdev as shown in Figure2.1. We refer to this as the anchor point of the ***INGRESS topology***. This location ID is also refered to as **ingress** in TC nomenclature. We are going to show later only the **ingress** qdiscs can be attached as the first first node on this anchor point.
- An ID of 0xFFFFFFF2 is reserved for use as a handle on the ingress anchor point of a netdev for the ***INGRESSCLSACT topology***. This is interchangeably used with **ingress** in TC nomenclature and offers the same semantics as *INGRESS* topology - for that reason we will only be discussing the INGRESS topology for the rest of this document.

3.2. TC Service Topology

The entry point for packets to a TC service topology (at an anchor point) is the qdisc FBT.

Packets flow into the ingress of a port towards the network stack or from the network stack egress towards a specific port. A TC service topology may be provisioned in either of those directions.

Starting with a qdisc, a selection of TC FBTs can be created and linked with control tools to create a network service topology. The root of a TC Service topology tree starts at one of the anchor ports described earlier (shown in Figure2.1).

3.2.1. Egress TC Service Topology

Figure2.2 shows a basic egress service topology (anchored on the egress of a port at handle 0xFFFFFFFF).



Figure2.2: EGRESS Classless Service Topology

As shown in Figure2.2 the root qdisc has a queue. Examples of qdiscs that fit this description include Pfifo, RED, etc. (more on this later). Only qdisc kinds that have queues can be anchored onto an egress service topology.

The qdisc (and therefore service) illustrated in Figure2.2 is basic. More complex egress service topologies can be built using qdiscs that are able to make use of classifiers pointing to classes. We refer to such kind of qdiscs as being *classful*. Examples of such qdiscs includes DRR, HTB, Prio, etc. Figure2.3 shows an example service topology which fits this description.

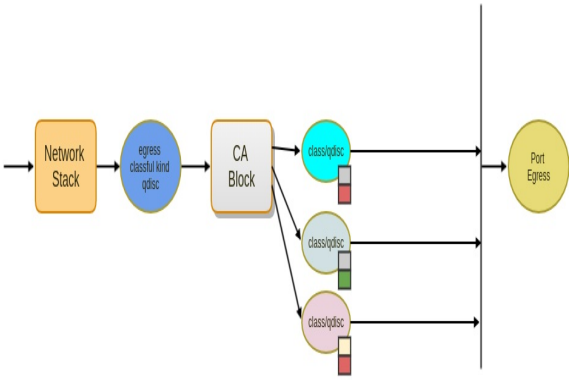


Figure2.3: EGRESS Classful Service Topology

As can be observed from Figure2.3, all the leaf qdiscs (encapsulated within their respective classes) on the service topology have queues.

Classful qdiscs can lead to other qdiscs as show in Figure2.4. With such a multi-level hierarchical qdisc setup, each hierarchy level requires demultiplexing to a selected class via a Classifier-Action (CA) block. It should also be noted that, like in Figure2.3, all leaf qdiscs have queues and qdiscs preceeding CA blocks have no embeded queues. We are going to refer to qdiscs preceeding CA blocks as ***branch qdiscs***.

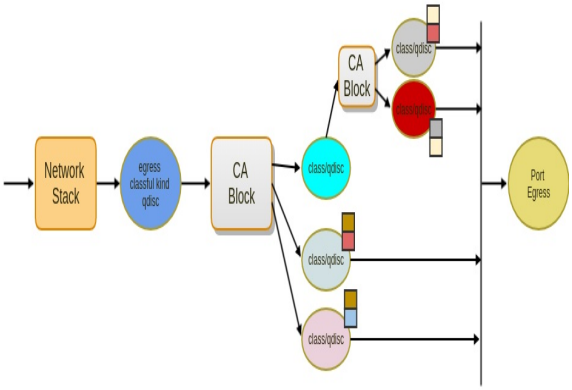


Figure2.4: EGRESS Complex Classful Service

3.2.2. EGRESSCLSACT TC Service Topology

There are many use cases where an egress service can be accomplished without the need for a qdisc with queues; in such a case, the branch qdiscs act as a point of demuxing to actions. To meet these egress service requirements, a qdisc kind *clsact* can be anchored at the egress of a port (with id 0xFFFFFFF3) and would serve only to hold CA block. Figure2.5 shows such a setup.

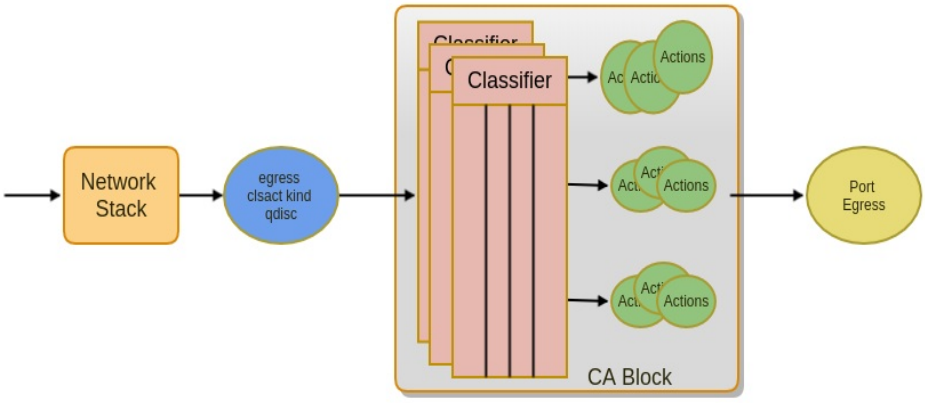


Figure2.5: EGRESS Clsact Service Topology

In Figure2.5 we zoom in on the CA block (shown earlier in Figure2.3 and Figure2.4) and illustrate how classifiers would multiplex to a graph of actions. We will discuss CA block in more details later on in this document.

XXX: Need to show both EgressCLSACT + Egress working together..

3.2.3. INGRESSCLSACT And Ingress TC Service Topology

On Ingress flow direction, a queue-less requirement is typically desired as well. Therefore, like the clsact qdisc kind, the Ingress qdisc kind does not own any queues. The Ingress qdisc kind is attached to the ingress of a port as shown in Figure2.6. Note as mentioned earlier, a clsact qdisc could also be attached to this point.

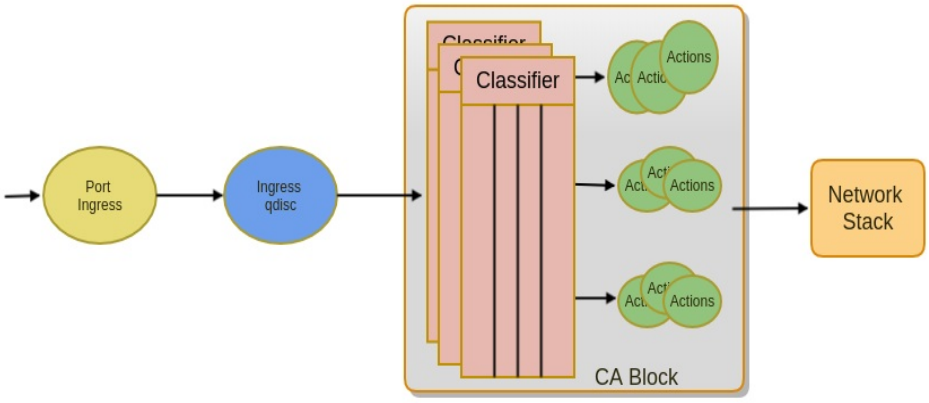


Figure2.6: INGRESS Service Topology

Figure2.2, Figure2.3, Figure2.4, Figure2.5, and Figure2.6 illustrate templates for possible service topologies. It should be emphasized that an instantiation of a service topology could use any of these templates and depending on parametrization of the different functional blocks, will result in unique services. In fact, it is possible to combine several of such topologies (ingress to egress or vice-versa) and to connect them together via the CA block - something we are going to discuss later in the document.

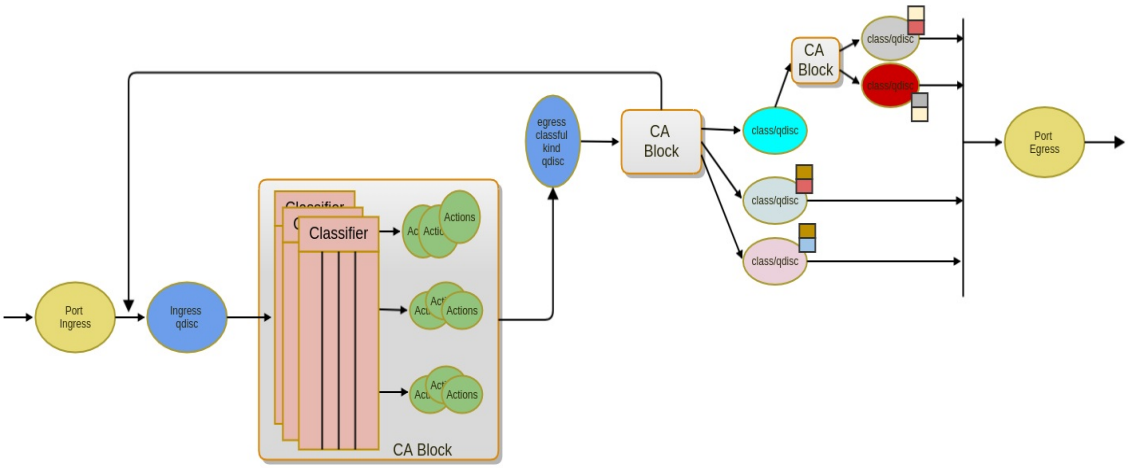


Figure2.7: INGRESS to EGRESS (Combined) Service Topology

3.3. Formal Definition Of Service Topology

To formalize the 3 possible topologies discussed so far, we try to define a BNF grammar in Listing2.1 to describe the different constraints that need to be accounted for:

- TOPOLOGY := INGRESS | EGRESSCLSACT | EGRESS
- INGRESS := `Ingress` CA
- EGRESSCLSACT := `clsact` CA
- EGRESS := QDISC-cf | `Qdisc`
- QDISC-cf := (CA CLASS+)
- CLASS := QDISC-cf | `Qdisc`
- CA := (`CLASSIFIER` `ACTION` +) +

Listing2.1: BNF Grammar

Notes on the BNF:

- `Ingress` is a qdisc of kind "ingress"
- `clsact` is a qdisc of kind "clsact"

- QDISC-cf is any classful qdisc
- `Qdisc` is a qdisc with one or more queues (example RED, HTB, etc)

A Service Topology **must** have a qdisc which may optionally have queues. The grammar shows three possible topologies: INGRESS, EGRESSCLSACT, and EGRESS. Neither INGRESS nor EGRESSCLSACT topologies have qdiscs with queues. INGRESS, as mentioned earlier, can only have a qdisc kind called Ingress (which is queue-less); and EGRESSCLSACT can only have the qdisc kind called clsact. The EGRESS topology always has queues associated with it.

A topology may have **optional** CA (Classifier-Action) blocks following the qdisc. Both EGRESSCLSACT and INGRESS topologies always have CA blocks; on the other hand, EGRESS topology will only have CA blocks if there is a presence of classful qdiscs (QDISC-cf). A classful qdisc is composed of a CA block followed by TC classes. A class encapsulates a qdisc which may be classful (`QDISC-cf`) or classless (`Qdisc`). A classless qdisc is a terminal/leaf in a service tree graph, whereas a classful one is a branching point in a service tree graph.

Classifiers are always followed by actions. Classifiers lookup either packet data or metadata and decide on the action graph branch to take. The action graph could constitute one or more actions that follow the classifier. An action could be a simple one, such as to select a class (in the EGRESS topology), or drop a packet, etc., but could also be complex and extend the service graph further by extending into new classifiers which will lead to another action sub-graph. We will discuss this further when we get into the details of actions.

The INGRESS topology is anchored at the ingress of a network port and, as described earlier, has at its root an ingress qdisc kind followed by a CA block. The ingress qdisc kind does not have any queues.

The EGRESSCLSACT topology is anchored at the egress of a network port and has at its root a clsact qdisc kind followed by a CA block. The clsact qdisc kind does not have any queues.

The EGRESS topology is also anchored at the egress of a network port (like EGRESSCLSACT) but constitutes qdiscs that have queues. Classful qdisc kinds lead to classes - allowing a complex topology to be created. Classes allow for adding branches to the topology. The branch is selected by a classifier within the topology. Classes have embedded qdiscs which may have *optional* provisioned CA blocks (if the qdisc is classful). The CA block further demultiplexes to another class in the provisioned topology - which begins a new topology sub-hierarchy. This pattern can be repeated ad-infinitum providing a fractal-like object layout until a terminal non-classful qdisc is reached.

For each of the 3 different topology types, packets arrive at a qdisc at the root of the provisioned graph hierarchy and traverses the topology downstream - consulting CA blocks at each branching qdisc to decide how to proceed onto the graph.

3.3.1. Document organization

In the rest of the document we will start discussing qdiscs first and then progress towards other FB types (Classes, classifiers/filters, and Actions). For each FB type we will describe both the topology location semantics and object identity and how they fit within a service topology.

XXX: this is the point where we say "in section 3 we will talk about .. and in section 4 .." etc.

4. Queueing Disciplines

A queueing discipline (qdisc) is a FBT centred around the concept of a queue. Two central qdisc methods are the *enqueue()* and *dequeue()* operations; each TC qdisc kind overrides these two methods with its own specific algorithm to achieve its end objectives. Example:

- The PFIFO qdisc will queue packets at the tail of an associated queue, whereas an active queue management algorithm like RED will first compute if it should queue the packet, drop it, or mark it, and then queue it.
- The ingress qdisc does not have a queue and uses the *enqueue()* method to invoke the CA block.
- The PFIFO qdisc will dequeue from the head of the queue, whereas a shaping qdisc (such as TBF) will first compute if the consumed bandwith associated with the queue is still within the range of configured bandwidth rate, and if it is, proceed to dequeue a packet, else refuse to.

As discussed earlier: a qdisc is uniquely defined with the tuples *{ifindex of netdev, parent id, handle id}* with the first two coordinates defining a topology location and the last one defining identity.

A TC qdisc instance is created by user control (using the iproute2 *tc* config utility[1] or programmatically from any other application using netlink APIs) with the following parameters:

- Device/netdev, e.g. "eth0" etc.
- Parent ID - note that the parent could be a netdev anchor point, as described in Figure2(with id 0xFFFFFFFF or 0xFFFFFFF1 etc), or another qdisc's handle when branching on classful qdiscs.
- Handle ID

Handle IDs are typically described (for readability) by *tc* using a **major:minor** numbering scheme, each 16 bit; we will be using the major:minor notation through out the document. A qdisc ID always has a minor value of 0. Later on when we describe classes, we will see how the minor number is used (and is non-zero).

In this section, we will discuss the egress topology since it is the only one that allows queues.

Figure3.1 illustrates a simple (non-classful) qdisc.

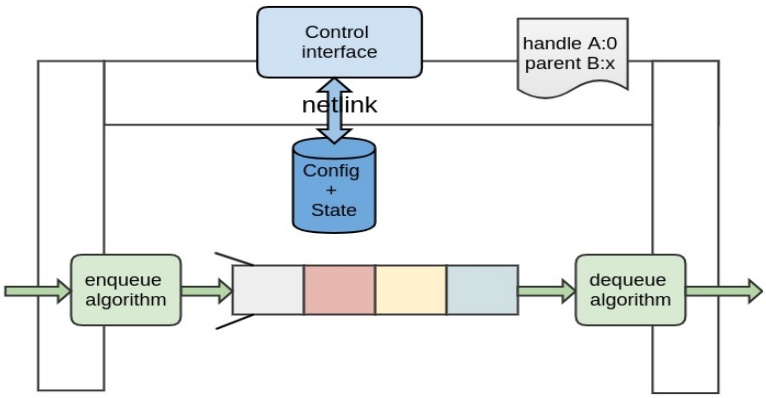


Figure3.1: Basic Egress Qdisc Setup

As shown, the qdisc has a handle (A:0) and a parent (B:x). The qdisc could be anchored to the egress of a device directly (Parent id being FFFF:FFFF or FFFF:FFF3) or maybe in a class in the hierachy (and a parent could be of the form 1:0).

Another detail to pay attention to in Figure3.1 is the fact that control (user space) has access to configuration and state of the qdisc instance via a netlink control interface (more on this later).

Last but not least, note that Figure3.1 highlights both the *enqueue* and *dequeue* operations for the qdisc instance. Figure3.1 shows presence of a queue of packets accessible to both operations. Depending on the qdisc kind and topology, a queue of packets may not exist at all. As an example:

- the blackhole qdisc *enqueue()* method just drops any packets handed to it (and has no real queues).
- the ingress qdisc *enqueue()* is used for holding the classifier-action block and has no queue whatsoever.

For illustration, let us show an example of creating a basic EGRESS topology service using a qdisc of kind `pfifo` to the egress of eth0. We will constrain the pfifo qdisc to have a maximum queue size to 1000 packets.

> ***sudo tc qdisc add dev eth0 parent root handle 1:0 pfifo limit 1000***

Listing3.1: Sample TC Egress Service Creation

Essentially the configuration in Listing3.1 has instantiated a qdisc kind `pfifo` with coordinates *{eth0, root, 1:0}*. Note: as discussed earlier, the term *root* in *tc* nomenclature implies the anchor point FFFF:FFFF (Figure2.1).

Furthermore, the instantiatiation has provided the *limit* attribute used by Pfifo qdisc kind to describe the maximum number of packets (*1000*) that can be queued onto the encapsulated queue.

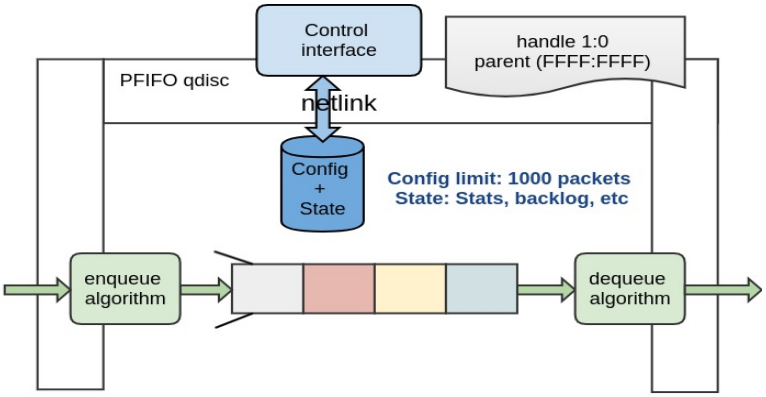


Figure3.2: Pfifo Qdisc Setup

Figure3.2 visualizes the configured service topology.

For completion, let's use the netlink interface to query the configuration and state of the instantiated qdisc.

> **tc -s qdisc ls dev eth0**

```
qdisc pfifo 1: root refcnt 2 limit 1000p
  Sent 0 bytes 0 pkt (dropped 0, overlimits 0 requeues 0)
  backlog 0b 0p requeues 0
```

Listing3.2: Egress Service Config And State

The results also show statistics associated with packets passing through this qdisc.

4.1. Topology Branching With Qdiscs

EGRESS service topologies can have hierachies as described in the the grammar in Listing2.1 as well as illustrated in Figure2.3 and Figure2.4. The branching feature is facilitated by qdisc kinds that we refer to as being `classful`. A classifier adjacent to a classful qdisc is used select a downstream qdisc (encapsulated within a class).

4.1.1. Classifier Types For Selecting Classes

To get clarity on branching we need to have a basic understanding of classifiers.

There are two types of classifiers in the TC infrastructure: `built-in` and `explicit`.

> 1. Built-in classifiers are omni-present adjacent to classful qdiscs. These classifier kinds use packet metadata for their selection criteria. The most common metadata inspected for classification purpose is the *skb->priority*; however, there are qdiscs which stare at the *skb->mark* for the demux decision. Built-in classifiers require no provisioning (they are embedded in the qdisc).
> 2. Explicit classifiers look at either or both of packet data and metadata. We will encounter a few examples of such classifiers (*u32*, *flower*, etc) in later sections. Explicit classifiers require user control provisioning.

We will discuss both types of classifiers later on. For now, suffice it is to say, we use the classifier to select classes.

4.1.2. Class Location And Identity

An instantiated TC *Class* object is essentially a "wrapper" of a qdisc object. The sole purpose of a *Class* is to provide a way for the EGRESS topology to bind qdisc objects (and thus enabling hierachical topology).

Just like qdiscs, TC classes have coordinates constituting a topology location and identity. The coordinate addressing is the same as that used by a qdisc: *{ifindex of netdev, parent ID, handle ID}*. The ifindex and parent ID identify a location in the service topology and the handle ID provides the identity of the class. The parentid of a class is always a qdisc. The class handle ID is derived from the same ID space as the qdiscs - with a minor exception: Class IDs make use of the minor number. Going forward we will interchangeably refer to Class handle IDs as ClassIDs.

To reiterate, class selection is achieved by way of a *classifier* within a topology. A *classifier* selects a *TC Class* (the similarity in the naming of these two TC components should give away their relationship ;->).

Figure3.3 shows a sample configuration.

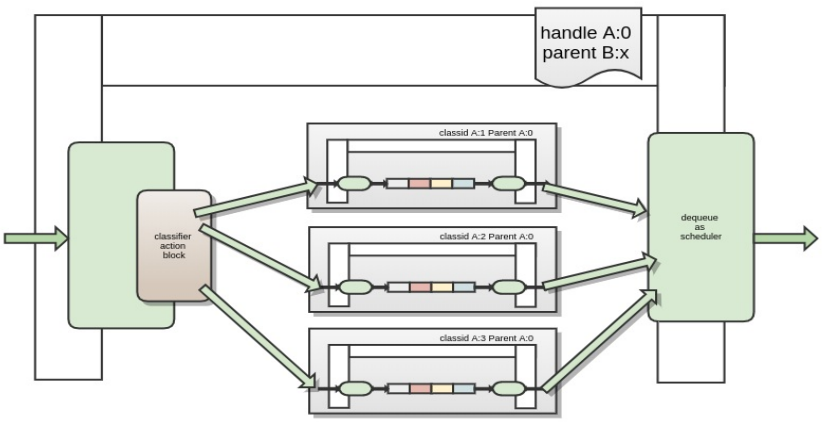


Figure3.3: Classful Qdiscs

In Figure3.3, we have a root node (ID A:0) attached to parent B:x. Downstream are three branch nodes A:1, A:2, and A:3.

When a packet arrives at the topmost qdisc's (handle A:0) *enqueue* method, the attached classifier uses either/or both packet data and/or metadata to select a class. The selected class' encapsulated (inner) qdisc's *enqueue* method is then invoked and handed the packet data and metadata. The encapsulated (inner) qdisc(FBT) kind will then apply its own algorithm (and the pattern repeats if the inner qdisc is classful and has classes).

The *dequeue* method implementation of a classful qdisc is typically a queue scheduling algorithm. Such an algorithm decides which queue to use as its source of a packet to send out downstream the service topology.

For illustration of classful qdiscs, let's instantiate a prio qdisc (which is classful) and attach it to the egress of eth0.

> ***$sudo tc qdisc add dev eth0 root handle 1: prio***

Listing3.3: Classful Egress Service

Figure3.4 demonstrates the resulting setup.

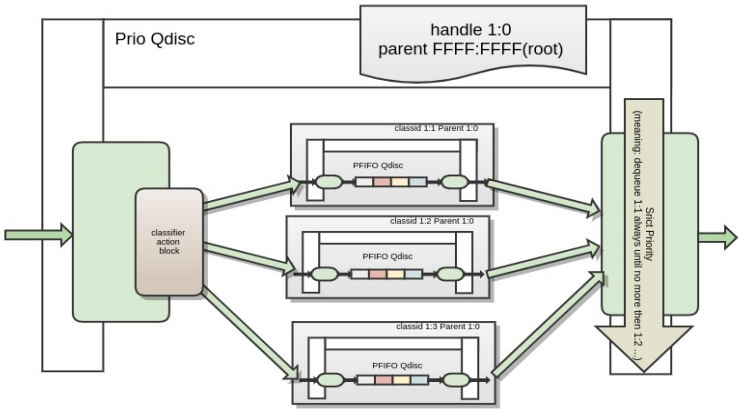


Figure3.4: Prio Qdiscs

Essentially the configuration in Listing3.3 instantiates a qdisc of kind *Prio* with coordinates *{eth0, root, 1:0}*. When a *Prio* qdisc is instantiated with no additional attributes specified, it creates 3 default classes - each encapsulating a qdisc of kind *pfifo*. The branch nodes 1:1 - 1:3 can be changed or replaced with other qdisc kinds. We will discuss this later.

A Prio qdisc by default uses a built-in classifier which uses the *skb->priority* metadata to select a class to which it enqueues to. To understand this let us query the configuration of the qdisc via netlink (using *tc*):

> ***$sudo tc qdisc ls dev eth0***

```
qdisc prio 1: root refcnt 2 bands 3 priomap  1 2 2 2 1 2 0 0 1 1 1 1 1 1 1 1
```

Listing3.4: Classful Egress Service Config View

The keywords "bands" (set to 3) and "priomap" define the scheduling approach taken. Bands are essentially queues - we see 3 above. Priority metadata to class selection is programmed in the "priomap".

Let's display the classes associated with this qdisc using *tc*.

> ***$sudo tc class ls dev eth0***

```
class prio 1:1 parent 1:
class prio 1:2 parent 1:
class prio 1:3 parent 1:
```

Listing3.5: Classful Egress Service Class View

A *Prio* qdisc kind's *dequeue* method implements a *strict priority* scheduler. Queue 0 - which is referred to as "band 0" - is mapped to classid 1:1 in Figure3.4; band 1 is mapped to classid 1:2 and band 2 is mapped to classid 1:3.

Classid 1:1 is considered the highest priority queue - always dequeued from whenever it has packets. Only when classid 1:1 has no more packets will the other classes be asked for packets by the scheduler using a strict priority: classid 1:2, next, and so on.

Heterogenous Qdiscs On EGRESS Topology

The TC EGRESS topology allows for different qdisc kinds to be placed in the different graph nodes. It is therefore possible to build complex network services (example: Assured Forwarding or Expedited Forwarding Service etc.).

To demonstrate the feature we build upon the already instantiated prio qdisc from Figure3.4 and replace its existing class default pfifo qdiscs in each of the 3 classes with other qdisc kinds.

> ***$ sudo tc qdisc add dev eth0 parent 1:1 handle 10: tbf limit 2000 rate 6mbit burst 15k***
>
> ***$ sudo tc qdisc add dev eth0 parent 1:2 handle 20: pfifo limit 500***
>
> ***$ sudo tc qdisc add dev eth0 parent 1:3 handle 30: sfq limit 255***

Listing3.6: Non-Homegenous Egress Service Creation

The above configuration replaces the qdisc in class 1:1 with a Token Bucket Filter (TBF) qdisc; packets coming out of class 1:1 will be shaped to 6 Mbps. Class 1:2 is replaced with a pfifo qdisc with a maximum queue size of 500 packets (replacing the default 1000) and class 1:3 qdisc, with Stochastic Fair Queueing (SFQ), limited to 255 packets. We will discuss what each of these algorithms implement later; for now, it is suffice to say that the TC architecture provides us flexibility to express topology of different FBT kinds.

To better appreciate the EGRESS topology operations, for the rest of this document we are going to switch to a slightly different visualization so we can focus more on illustrating the hierarchical abstraction. Figure3.5 uses this new illustration approach to visualize the setup derived from Figure3.4 that is described above.

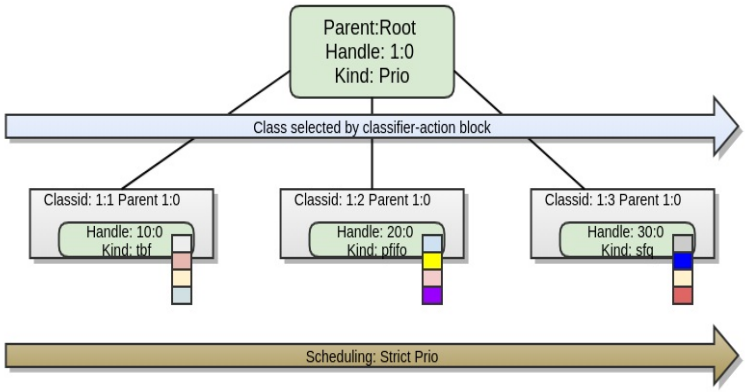


Figure3.5: Alternate Prio Qdisc

Let's query the qdisc setup we created for state and config:

> ***$sudo tc -s qdisc ls dev eth0***

```
qdisc prio 1: root refcnt 2 bands 3 priomap  1 2 2 2 1 2 0 0 1 1 1 1 1 1 1 1
 Sent 0 bytes 0 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
qdisc sfq 30: parent 1:3 limit 255p quantum 1514b depth 127 divisor 1024
 Sent 0 bytes 0 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
qdisc tbf 10: parent 1:1 rate 6Mbit burst 15Kb limit 2000b
 Sent 0 bytes 0 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
qdisc pfifo 20: parent 1:2 limit 500p
```

```
  Sent 0 bytes 0 pkt (dropped 0, overlimits 0 requeues 0)
  backlog 0b 0p requeues 0
```

Listing3.7: Non-Homegenous Egress Service Config And State

Observe the config of the different qdiscs in Listing3.7 - note that the kernel will provide some good defaults for the different attributes, which we did not specify, for each of the qdisc creations we provisioned in Listing3.6 (example: sfq creates 1024 hash buckets, etc.). Each of these attributes can, of course, be user specified (instead of depending on kernel defaults).

Let's query the classes:

> ***$sudo tc -s class ls dev eth0***

```
class prio 1:1 parent 1: leaf 10:
 Sent 0 bytes 0 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
class prio 1:2 parent 1: leaf 20:
 Sent 0 bytes 0 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
class prio 1:3 parent 1: leaf 30:
 Sent 0 bytes 0 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
class tbf 10:1 parent 10:
```

Listing3.8: Non-Homegenous Egress Service Class Config And State

It should be noted that the TBF qdisc also spawns one class: classid 10:1 (just like the Prio qdisc did). This is a consequence of how the kernel sets TBF - although it is still a single class it provides opportunity to replace the embedded qdisc with some other qdisc kind (example: we could replace the pfifo with a Prio qdisc enabling further hierarchies).

Selecting Classes Using Built-In Classifier

As was discussed earlier, classifiers can be either built-in or explicit. In this section we discuss built-in classifiers. There are two approaches to built-in classifiers: 1) direct mapping and 2) qos mapping.

Each qdisc kind may approach the built-in classifier differently in terms of which metadata to use for branching, which of the two above approaches to implement, or when implementing both approaches, how to prioritize one over the other. We will describe the prio qdisc approach since it encompasses both built-in approaches using the `skb->priority` metadata.

As mentioned earlier, the Prio qdisc has a built-in classifier which looks at the priority metadata field (`skb->priority`) and selects one of the classes/queues based on the metadata value. The packet's priority metadata value could be set by different parts of the kernel before an egress prio qdisc sees it; for example: the Vlan code transfers the 802.1p value to it; the IPV4 routing code copies the TOS fields to this same field (XXX: Is it TOS still?); a user could (running a server/client) set this value with a socketsetopt() call; and a CA block at any topology (ingress etc) could set it via the skbedit action.

When the Prio qdisc *enqueue()* method is invoked, it uses the priority metadata field value to index into the priomap array shown in the configuration earlier (1 2 2 2 1 2 0 0 1 1 1 1 1 1 1 1). We will call this the `qos mapping` approach for priority classification. There are 16 possible priority values (0-15) in the above default priomap. When a value of zero is seen on the packet, the prio entry at priomap[0] (value = 1) is found. The value 1 maps to class 1:2. When prio is 2, index 2 (priomap[2]) is looked up and the resulting value of 2 is mapped to 1:3, etc. When a priority goes outside the acceptable range (0-15 in the above priomap), the index 0 is used to select the class (classid 1:2).

There is another way the priority metadata is used to classify; to directly map the metadata to a classid. We will call this the `direct mapping` approach. For example, a socket user or skbedit action could set the value to 0x00010003 which is equivalent to 1:3. When the Prio qdisc *enqueue()* is provided such a value

it selects class 1:3 directly. Almost all the classful qdiscs (ATM, CBQ, DRR etc) support this direct mapping classification method.

4.1.3. Dequeueing From Qdiscs

As mentioned earlier, the *dequeue()* method is used by different qdisc kinds to implement code that suits that qdisc kinds's algorithm end goal. Essentially in the cases of qdiscs with queues, this is tantamount to scheduling algorithms, i.e. in what order packets should be scheduled out and when they should be scheduled out.

In the case of a classful qdisc, the associated dequeue algorithm typically defines scheduling of the enqueued packets; so as to make decisions when those packets depart the qdisc (if they do at all).

For the sake of simplifying the concept description, we are going to assert that the dequeue workflow as something initiated by the kernel. The kernel starts by invoking the service topology root qdisc *dequeue()* method. The root node in turn invokes its downstream class/qdisc *dequeue()* methods. The downstream qdisc *dequeue()*, depending on its algorithm, may or may not return a packet even if one was available on its queue. As an example, a shaping qdisc like TBF will not return a packet when its *dequeue* method is invoked if the shaping rate is exceeded for that queue.

Let us look at the prio qdisc to illustrate classful dequeueing.

The prio qdisc implements a strict `work conserving priority scheduling algorithm`[2]. The highest priority queue (classid 1:1) is serviced until it is empty; then classid 1:2 is dequeued from until it is empty (but if at any point a packet shows up on classid 1:1, then it is serviced next since that queue takes precedence); and lastly classid 1:3 is serviced (unless classid 1:2 or 1:1 have packets in which case servicing those queues takes precedence). As can be observed, this means that in the default setup, classid 1:1 can starve out classid 1:2 and 1:3, and likewise classid 1:2 can starve 1:3. *This is the design intent of the Prio scheduler*.

In the default prio setup we have shown so far in Figure3.4, all the classes encapsulate pfifo qdiscs. The user could choose to change the pfifo qdisc to alleviate the starvation of lower priority queues. In fact the example in Figure3.5 does just this by replacing the default pfifo qdisc with TBF, which rate limits the dequeueing of classid 1:1 to 6Mbps. When the classid 1:1 *dequeue()* is invoked by parent qdisc's (1:0) *dequeue()*, it will always return a packet - unless the rate of 6Mbps is exceeded; otherwise, the behavior is as if that queue is empty, in which case the Prio qdisc scheduling gives a chance to classid 1:2 to send something.

5. Complex EGRESS Topologies

Let us build a topology with two levels of hierarchies to demonstrate something more complex:

```
$ sudo tc qdisc add dev eth0 parent root handle 1: drr
$ sudo tc class add dev eth0 parent 1: classid 1:1 drr
$ sudo tc class add dev eth0 parent 1: classid 1:2 drr
$ sudo tc class add dev eth0 parent 1: classid 1:3 drr
$ sudo tc qdisc add dev eth0 parent 1:2 handle 20: prio
```

Listing4.1: More Complex Egress Topology Creation

At the top of the hierarchy is a Deficit Round Robin (DRR) qdisc anchored at handle 0xFFFF (root) with three classes. DRR class 1:2 extends the topology based on the prio qdisc. The provisioned setup is visualized in Figure4.1.

Let's see the qdisc config.

```
$ sudo tc qdisc ls dev eth0
  qdisc drr 1: root refcnt 2
  qdisc prio 20: parent 1:2 bands 3 priomap  1 2 2 2 1 2 0 0 1 1 1 1 1 1 1 1
```

Listing4.2: More Complex Egress Topology Config

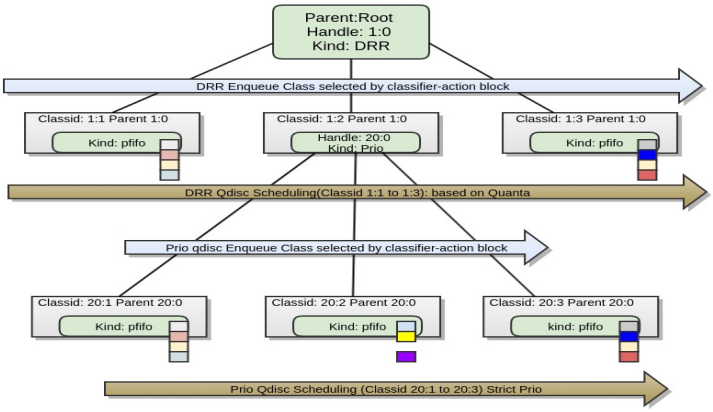


Figure4.1: More Complex Hierarchy

Let's see the class configuration.

```
$ sudo tc class ls dev eth0
class drr 1:1 root quantum 1514b
class drr 1:2 root leaf 20: quantum 1514b
class drr 1:3 root quantum 1514b
class prio 20:1 parent 20:
class prio 20:2 parent 20:
class prio 20:3 parent 20:
```

Listing4.3: More Complex Egress Class Topology Config

DRR by default uses the built-in priority metadata classifier to select classes on which packets are to be enqueued to. On the dequeue phase, DRR gives an opportunity for each queue to supply up to a quantum of bytes (1514 in the examples in Listing4.3).

5.1. Enqueuing On Complex Egress Topologies

Let us demonstrate how a packet would traverse the topology depicted in Figure4.1 for the purpose of enqueueing.

Let us assume that the packet arrives at the root DRR qdisc with `skb->priority` equal to 0x00010002. The sequence of events is as follows:

1. First the root (DRR) `enqueue()` method is invoked.
2. The `enqueue()` method consults built-in priority classifier - it uses the `skb->priority` as input to `direct mapping` classification. Class 1:2 matches. This class happens to encapsulate a prio qdisc with id 20:0.
3. The Prio qdisc 20:0 `enqueue()` method is invoked and consults its built-in classifier

   > - `direct mapping` classification will fail to find a match because qdisc 20:0 does not have a child class with id 1:2;
   > - `qos mapping` classification is consulted next and will fail since the value 0x00010002 is above the maximum priority value of 15. The prio qdisc in this case will assume a priority value of zero as the default. `qos mapping` classification is used indexing into the priomap array index 0 to map to class 20:2.

4. The packet is enqueued on the queue encapsulated within class 20:2.

5.2. Dequeuing From Complex Egress Topologies

Let us demonstrate how a packet would be dequeued from the topology depicted in Figure4.1.

1. The root DRR `dequeue()` method is invoked
2. The root `dequeue()` method invokes classes 1:1 to 1:3 in that order. For each of the classes:
   - Invoke encapsulated `dequeue()` methods (pfifo for class 1:1 and 1:3 and prio for class 1:2).
     - Each of the child classes is allowed to provide up to a quanta size (initally shown as 1514 bytes in the config above). The class is requested to provide data for as long as the sum of the bytes for that request round does not exceeded the available quanta.
     - If the class is not able to provide the full quanta amount of bytes, then whatever the difference is gets added to its quota for the next time. Example if a class was only able to provide 514 bytes when it had a quanta of 1514 bytes, the next time it will be allowed to send up to 2514 bytes.
     - If the class has more data than the quanta allows, then it will miss its opportunity to transmit but will get a better opportunity the next time. As an example, lets say the class had only 514 bytes quanta available and had a 1000 bytes packet to send, it will not be allowed to send at that moment but will get its quanta incremented by 1514 bytes for the next round (for a total of 2028 bytes).

Note: Class 1:2 being hierarchical, is treated by DRR the same as the other 2 classes, i.e. it will be allowed to send 1514 bytes. Given prio is in charge of class 1:2 then that class is subject to prio algorithm when it gets requested to dequeue a packet. It is up to the prio qdisc to pick which of its associated classes to dequeue from; and as we have seen, it will dequeue from class 20:1 always unless class 20:1 has no more packets etc. DRR is totally oblivious to this - all it is requesting from the prio qdisc is to be provided up to a quanta amount of bytes.

6. Classifiers

We already talked about implicit classifiers earlier. In this section we will describe explicit classifiers a lot more and go into more details in regards to the Classifier *FBT*.

6.1. Using Explicit Classifiers

As already mentioned, classifiers are one of the key functional block types in the TC architecture. Their role is to look at input packet data and/or metadata and decide what treatment a packet gets. So far we have looked at the built-in classifier which operates on the skb->priority metadata. We are now going to introduce the second kind of classifiers that we are going to refer to as `explicit classifiers`. Explicit classifiers are user provisioned. There are many kinds of explicit classifiers (just like there are many kinds of qdiscs) all of which are user plug-able into the service topology. The underlying architectural decision to allow different algorithms is based on the fact that **it is impossible to have one classifier algorithm that is optimal for all kinds of packet meta/data classification**. Some examples:

- A match on a string using the latest string search algorithm is most likely not the most fit algorithm for IPv6 header classification.
- Even for the same type of field/tuple classification (e.g. looking up destination IP addresses), some algorithms are better suited for specific environments than others; for example, small systems with few rules care more about RAM cost for policy storage compared to a large system where lookup speed is more important than cost of storage, etc.

Given that classifier algorithms are a constant source of industry advancements and academic curiosity, the idea that a new classifier could be authored and easily plugged-in makes the TC architecture very flexible.

There are many kinds of classifiers in the TC subsystem. Here are a few that work on packet data :

- u32, which matches based on 32-bit ternary key/mask chunks on arbitrary packet offsets. U32 is also capable of looking up rules based on skb->mark metadata in addition to the packet data.
- flower, which matches based on many pre-defined header tuples (src/dst ipv4/6, ip protocol, etc.) to match on packets. On the egress side it makes use of the packet flow information already constructed by the network stack to speed up its lookups.
- bpf, which matches via arbitrary EBPF programs logic on packets. It is up to the EBPF program author to decide on the algorithm.

And here are a few samples that are metadata based:

- cgroup, which matches on control groups (cgroups) metadata assignment on packets.
- fw, which matches on skb->mark metadata
- route, which matches based on *route realm* metadata added by the routing subsystem

User control can create an instance of a classifier referred to as a filter. Filters of different classifier kinds can be grouped together into a list known as a chain. To resolve ambiguity in a chain, filters are ordered by ascending preference; i.e. when more than one filter rule matches a packet, the highest priority filter is chosen as the least ambiguous.

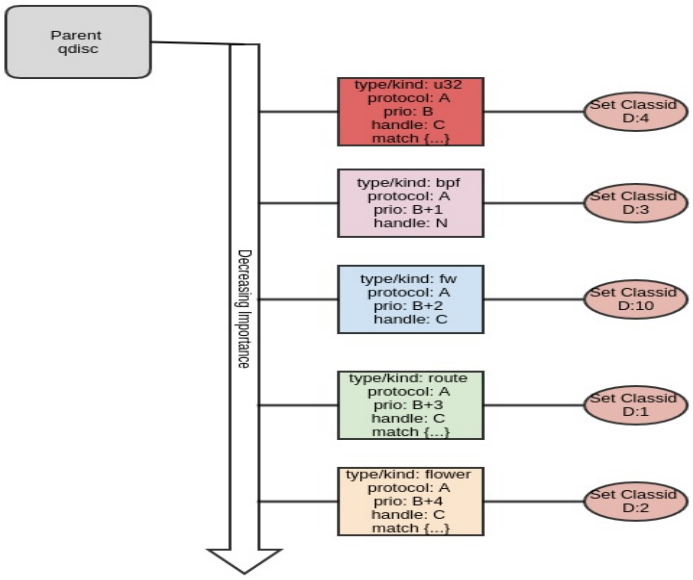


Figure6.1: TC Filter Chain

Figure6.1 shows a sample of a filter chain.

Filter entries in a chain are *keyed by ethernet protocol* (IPv4, IPV6, ARP, etc.). Further there's a loose rule which is important to mention: *Filters with the same protocol on the same chain must have different priorities* with some very special exceptions which we will discuss later (u32 filters in a table XXX which may have the same priority).

6.1.1. Filter Topology Location

As was described in section XXXX, the qdisc enqueue() method is the point in which classifiers are invoked for the purpose of selecting classes. Filter chains must therefore be installed in the service topology at locations where classful qdiscs reside. Figure6.1 illustrates an example where a filter chain is attached to a qdisc.

The basic coordinate to consider for a filter follows the same semantic for location as a qdisc/class; i.e. *{netdev, parent ID}* and as mentioned earlier, location is further refined by *{protocol, priority}*.

6.1.2. Filter Identity

Most of the time it is sufficient to specify the coordinates: *{netdev, parent ID, protocol, priority}* to uniquely identify a filter with some exceptions which we will discuss.

Figure6.1 is a little deceiving - it gives the impression that the lookup is a linked list of filter entries. In fact, depending on the classifier, a filter on a chain can have a reference to a table or even multiple hash tables (the u32 classifier for example is one such classifier). To uniqueuely identify a filter, the TC subsystem uses a 32-bit filter *handle ID*. Note, this is similar to the handle ID that identifies a class or a qdisc. There are two differences compared to qdisc or class handles; in that filter handle semantics are per classifier-type specific:

- filter handles IDs are tied to the classifier kind semantics. In some cases a filter handle could map to a table index; in others a filter handle maps to a hash key not different from a TCAM key, etc. Given the handle is filter-kind specific, it is, in fact, possible to have the same filter handle ID amongst filters in a chain (as long as they are of different classifier-kinds). It may even be possible to have the same exact filter id for many different priorities of the same classifier kind when the handle semantic is unneeded (e.g. the flower classifier does this).
- filter handle IDs are sometimes required to be specified by the user control (e.g. the fw classifier

requires it) and in other cases are issued by the kernel but could be optionally provided by user control (e.g. u32 and flower classifiers behave this way).

XXX: Refer to netdev01 and netdev11 papers which describe u32 to some extent (and when the doc grows to be very large we can go into the details).

The summary is: the user may need to understand the classifier kind to make sense of the filter handle.

Let us show an example by installing a flower filter on the DRR qdisc of Figure4.1:

```
$ sudo tc filter add dev eth0 parent 1:0 protocol ip priority 10 \
flower dst_mac 52:54:00:3d:c7:6d \
classid 1:1
```

Listing6.1: Filter Creation

The filter instantiated from classifier kind flower uses the flower classification algorithm to match a destination mac address of *52:54:00:3d:c7:6d*. The filter location (and identity in this case) in the tuples *{netdev, parent ID, protocol, priority}* are set as *{eth0, 1:0, ip, 10}*. The flower classifier does not require to set the filter *handle* identification as the kernel will provide one for us.

The kernel makes some default choices when a user creates filters; when priority is not specified, the kernel will choose a very low priorty. Additionally, if the parent is not specified, egress anchor point (0xFFFFFFFF) is assumed.

Let's display the kernel's view of what we installed:

```
$ sudo tc filter ls dev eth0
filter parent 1: protocol ip pref 10 flower chain 0
filter parent 1: protocol ip pref 10 flower chain 0 handle 0x1 classid 1:1
 dst_mac 52:54:00:3d:c7:6d
 eth_type ipv4
 not_in_hw
```

Listing6.2: Filter Configuration

Observe that the kernel provisioned the handle of 0x1 as the filter id. For the flower classifier, the handle is not that important for completion, however, we note the filter location and identity above is: *{netdev, parent ID, protocol, priority, handle}* being set as *{eth0, 1:0, ip, 10, 0x1}*.

Filter Chains

Figure6.1 shows a generic example of a filter chain. It shows multiple filters derived from different classifier kinds (bpf, flower, u32, fw, route). Each filter rule matches certain meta/data headers and sets the 32 bit classid (tc syntax "classid X:Y") to select a DRR class to enqueue the packet to.

Let us add a second lower priority filter using the u32 classifier to match all ICMP packets on the example setup from Figure4.1 in Listing6.3.

```
$ sudo tc filter add dev eth0 parent 1:0 protocol ip priority 11 \
u32 match ip protocol 1 0xff \
classid 1:2
```

Listing6.3: Filter Chain Addition

When the filter matches, we select classid 1:2 as the target class to enqueue on. We visualize this in Figure6.2:

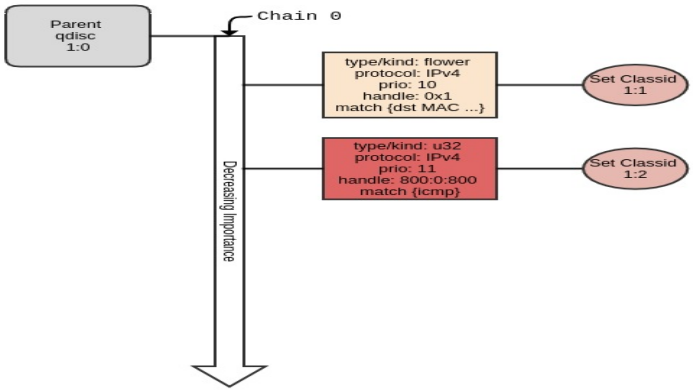


Figure6.2: Simple TC Filter Chain At Parent 1:0 Location

So what happens to a packet arriving on the DRR qdisc at the root anchor?

- The enqueue() method consults the built-in priority classifier - it uses the `skb->priority` as input to `direct mapping` classification. If this metadata value matches a known class, then the packet is enqueued to the correct class. Else:
- if it is an IPV4 packet, then:
  - if the packet carries a destination MAC address of 52:54:00:3d:c7:6d, it will match the flower rule and will be enqueued to class 1:1 via that class' pfifo enqueue(); else the next priority filter in the chain (u32 rule below) is consulted.
  - if it is an ICMP packet, the u32 rule matches, class 1:2 enqueue() is invoked, and it ends up in the prio qdisc 20:0 using the built-in classifier as mentioned earlier in XXX.
  - else it failed to match - the DRR scheduler drops the packet.
- else it is not an IPv4 packet, and the DRR scheduler drops the packet.

Let's dump the filter chain configuration:

```
$ sudo tc filter ls dev eth0
filter parent 1: protocol ip pref 10 flower chain 0
filter parent 1: protocol ip pref 10 flower chain 0 handle 0x1 classid 1:1
 dst_mac 52:54:00:3d:c7:6d
 eth_type ipv4
 not_in_hw

filter parent 1: protocol ip pref 11 u32 chain 0
filter parent 1: protocol ip pref 11 u32 chain 0 fh 800: ht divisor 1
filter parent 1: protocol ip pref 11 u32 chain 0 fh 800::800 \
 order 2048 key ht 800 bkt 0 \
 flowid 1:2 not_in_hw
  match 00010000/00ff0000 at 8
```

Listing6.4: Filter Chain Config And State

As can be observed, the two filters are ordered according to their provisioned priorities. The u32 filter specifies a filter handle of 800::800 (or 0x80000800); as mentioned earlier this is very specific to the u32 way of describing the 32 bit handle id. We will discuss the semantics of u32 filtering later. Also note that the u32 tc output shows a term "flowid"; we will interchangeably use this term with "classid" throughout the document.

Let's say it turns out to be a bad idea to drop packets that failed to match; we add another rule to "match everything else" and send it to class 1:3. For fun, let us use a new simple classifier kind called *matchall* (any of the other classifier kind could have been used, but matchall exists merely to match everything, so seems ideal to use).

```
$ sudo tc filter add dev eth0 parent 1:0 protocol ip priority 16 \
```

```
  matchall \
  classid 1:3
```

Listing6.5: Filter Chain Default Rule Config

And dumping the filter chain again:

```
$ sudo tc filter ls dev eth0 chain 0
filter parent 1: protocol ip pref 10 flower
filter parent 1: protocol ip pref 10 flower handle 0x1 classid 1:1
  dst_mac 52:54:00:3d:c7:6d
  eth_type ipv4
  not_in_hw

filter parent 1: protocol ip pref 11 u32
filter parent 1: protocol ip pref 11 u32 fh 800: ht divisor 1
filter parent 1: protocol ip pref 11 u32 fh 800::800 \
 order 2048 key ht 800 bkt 0 \
 flowid 1:2 not_in_hw
  match 00010000/00ff0000 at 8

filter parent 1: protocol ip pref 16 matchall
filter parent 1: protocol ip pref 16 matchall handle 0x1 flowid 1:3
  not_in_hw
```

Listing6.6: Filter Chain Config And State

To visualize this new configuration from Listing6.5, it will look as shown in Figure6.3:

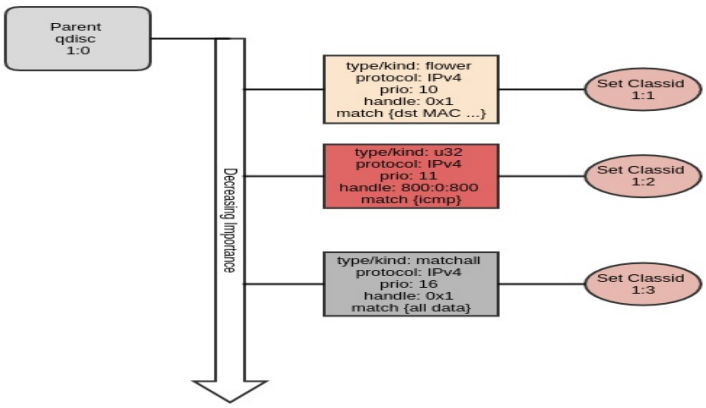


Figure6.3: Simple 3 Rule TC Filter Chain At Parent Location 1:0

With the new setup, any packet not matching the flower or u32 rules will be matched by the matchall rule at priority 16 - and therefore no packets will get dropped.

Topology Class selection

Although we have thus far shown filter chains being adjacent to classful qdiscs selecting classes adjacent to them in the topology, a filter can, in fact, select a class that is on a totaly different topology branch. As an example, in Listing6.7 we extend the setup in Figure6.3 and add a u32 rule to the filter chain at location 1:0 to match src subnet 10.0.0.0/24 and select it to be enqueued on classid 20:3 (i.e. not one of the adjacent 1:1-1:3 classes), which is in a downstream branch location in the topology.

```
$ sudo tc filter add dev eth0 parent 1:0 protocol ip priority 12 \
  u32 \
  match ip src 10.0.0.0/24 \
  classid 20:3
```

Listing6.7: New Filter Creation

Let's display the filter chain config at that level:

```
$ sudo tc filter ls dev eth0 parent 1:0 chain 0
  filter protocol ip pref 10 flower
  filter protocol ip pref 10 flower handle 0x1 classid 1:1
    dst_mac 52:54:00:3d:c7:6d
    eth_type ipv4
    not_in_hw

  filter protocol ip pref 11 u32
  filter protocol ip pref 11 u32 fh 800: ht divisor 1
  filter protocol ip pref 11 u32 fh
  800::800 order 2048 key ht 800 bkt 0 flowid 1:2 \
   not_in_hw
    match 00010000/00ff0000 at 8

  filter protocol ip pref 12 u32
  filter protocol ip pref 12 u32 fh 801: ht divisor 1
  filter protocol ip pref 12 u32 fh 801::800 \
   order 2048 key ht 801 bkt 0 flowid 20:3 \
   not_in_hw
    match 0a000000/ffffff00 at 12

  filter protocol ip pref 16 matchall
  filter protocol ip pref 16 matchall handle 0x1 flowid 1:3 \
    not_in_hw
```

Listing6.8: Filter Chain Config And State

Filter Chains On Multiple Topology Locations

Figure6.2 and Figure6.3 show examples of a filter chain located at parent location 1:0. But as mentioned earlier, we can install these rules at different locations as prescribed by the filter rule coordinates.

As an example, on Listing6.9 we install a filter rule on the prio qdisc with handle id 20:0 (inside class 1:2) in the topology of Figure4.1.

```
$ sudo tc filter add dev eth0 parent 20: protocol ip priority 10 \
  u32 \
  match ip src 192.168.8.0/24 \
  classid 20:1
```

Listing6.9: New Filter Chain Creation

The filter rule selects class 20:1 for IPv4 packets matching IP source subnet 192.168.8.0/24.

Dumping the new filter chain config:

```
$ sudo tc filter ls dev eth0 parent 20:
  filter protocol ip pref 10 u32 chain 0
  filter protocol ip pref 10 u32 chain 0 fh 800: ht divisor 1
  filter protocol ip pref 10 u32 chain 0 fh 800::800 \
   order 2048 key ht 800 bkt 0 flowid 20:1 \
   not_in_hw
    match c0a80800/ffffff00 at 12
```

Listing6.10: New Filter Chain Config

XXX: May need to describe how a packet gets treated here.. **YES**

XXX: May need a diagram? **YES**

Basic Actions

So far we have seen that when a filter matches we can execute a "select a queue/class" action ("classid X:Y").

The TC Classifier-action block provides a lot more actions and an architecture that allows composition of a complex graph of actions when a packet is matched. As an introduction we will demonstrate a few basic actions.

Let's add an action on parent 20:0 to count packets coming from source IP 192.168.8.1 and then enqueue them on classid 20:2 in Listing6.11:

```
$ sudo tc filter add dev eth0 parent 20: protocol ip priority 9 \
  u32 \
  match ip src 192.168.8.1/32 \
  classid 20:1 \
  action ok
```

Listing6.11: New Filter Chain Extension

The action "ok" accepts all packets and accounts for the packets and their bytes. Notice how we strategically placed this rule before the rule which matches the whole subnet by setting a lower priority.

Let's add another action on parent 20:0 to "drop" packets coming from source IP 192.168.8.32/32

```
$ sudo tc filter add dev eth0 parent 20: protocol ip priority 8 \
  u32 \
  match ip src 192.168.8.32 \
  classid 20:1 \
  action drop
```

Listing6.12: New Filter Chain Extension

And now let's dump the filter chain config at parent 20:

```
$ sudo tc filter ls dev eth0 parent 20: chain 0

  filter protocol ip pref 8 u32
  filter protocol ip pref 8 u32 fh 802: ht divisor 1
  filter protocol ip pref 8 u32 fh 802::800 order 2048 key ht 802 bkt 0 \
   flowid 20:1 not_in_hw
    match c0a80820/ffffffff at 12
    action order 1: gact action drop
     random type none pass val 0
     index 2 ref 1 bind 1

  filter protocol ip pref 9 u32
  filter protocol ip pref 9 u32 fh 801: ht divisor 1
  filter protocol ip pref 9 u32 fh 801::800 order 2048 key ht 801 bkt 0 \
   flowid 20:1 not_in_hw
match c0a80801/ffffffff at 12
    action order 1: gact action pass
     random type none pass val 0
     index 1 ref 1 bind 1

  filter protocol ip pref 10 u32
  filter protocol ip pref 10 u32 fh 800: ht divisor 1
  filter protocol ip pref 10 u32 fh 800::800 order 2048 key ht 800 bkt 0 \
   flowid 20:1 not_in_hw
match c0a80800/ffffff00 at 12
```

Listing6.13:New Filter Chain Config And State

XXX: Build this up as an argument to talk about classifier-action blocks and queue-less qdiscs..
^ **YES** follow up with something out of P4 presentation

7. Classifier Action Blocks

In this section go into more details on classifiers action blocks.

- XXX: Multi-chains, blocks etc.
- XXX: location/identity address of classifiers..
- XXX: Describe relationship to Actions
- XXX: Describe relationship to chains/blocks

7.0.1. Actions

XXX: We go into details of Action topology etc…

7.0.2. Multiple Filter Tables

Talk about using u32 parsing and then using multiple filter chains

7.0.3. Shared Classifier-Action Blocks

XXX

8. Hardware Offloading

XXX: Talk about how we offload qdiscs, filters and actions

9. Kernel + User Code Overview

We go into code details

9.0.1. Qdisc Ops

XXX: Go into kernel level qdisc_ops structure XXX: Go into the netlink messages to configure the qdiscs

9.0.2. Class Ops

XXX: Go into kernel level class_ops structure XXX: Go into the netlink messages to configure the classes

9.0.3. Filter Ops

XXX: Go into kernel level filter_ops structure XXX: Go into the netlink messages to configure the filters

9.0.4. Action Ops

XXX: Go into kernel level action_ops structure XXX: Go into the netlink messages to configure the actions

10. Acknowledgements, Footnotes And References

10.1. Acknowledgements

This document was created using gitbook.

10.2. Footnotes

> [1]. The tc utility (part of iproute2), which uses the netlink interface to talk to the kernel, will be used through out this document to demonstrate access to state and config. To avoid ambiguity, going forward in this document, when we mention tc (in lower case) the reference is going to be explicit to the iproute2 utility; when we mention TC (upper case variant) we would be explicitly refering to the Linux Traffic Control architecture. ↩

> [2]. A scheduler that tries to keep the scheduled resource(s) busy. ↩

[3]:To all your coders out there ;-> These are merely abstractions and should not be mapped to exact implementations. Example, while TC actions can be abstracted using this concept, the implementation is very different.